

Predator_2

Premise Selection over ZFC Set Theory

What a Prover Must Choose Among Forty Thousand Theorems

Version 1.0

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

Predator2-ZFC-Brian_Tenneson_1.0 | July 27, 2026

Companion program: `predator2.py`

Copyright © 2026 Brian Tenneson. All Rights Reserved.

<https://btenneson2301.substack.com>

Abstract

Predator_1 proves theorems in a propositional Hilbert system by forward search: it applies rules, generates consequences, and is measured against brute-force closure in expansions. That design does not transfer to ZFC. Re-deriving a proof in Metamath’s `set.mm` requires the full substitution machinery of the language, and the corpus holds roughly 43,900 proved theorems over thousands of symbols; forward enumeration is not available at that scale.

What `set.mm` does supply, written in the file, is premise structure: every proof names the axioms and earlier theorems it invokes. Predator_2 therefore answers the question a prover working inside a large library actually faces — *which of the available theorems does this goal’s proof use?* — and brute force becomes reading the library in order. Effort is the fraction of the candidate pool one must examine before every true premise has been found, which is the premise-selection analogue of expansions and admits the same two-number scoring: recall, because a selector returning nothing has zero effort, and effort, because a selector returning everything has perfect recall.

The measurements reported here were made on a *structural stand-in*, not on ZFC. No network was available when this was written, so a corpus was generated with `set.mm`-like statistics — power-law citation, topical vocabulary, premise counts in the observed range — to exercise the pipeline. On it Predator_2 reaches recall@50 of 0.658 against 0.155 for a frequency baseline and examines 0.22 of the pool against brute force’s 0.80, a $3.7\times$ reduction. These numbers describe the harness. They are reported because the harness is what was built and tested; the mathematics is one download away, and Section 5 gives the command.

Keywords: premise selection, ZFC, Metamath, `set.mm`, automated theorem proving, temporal split, recall, search effort, tagged ATP.

Contents

1 Why the task changes	3
2 The prover	3
3 Scoring	3
4 Measurements on a structural stand-in	4
5 Running it on ZFC	4
6 Scope	5
Acknowledgements	5

1 Why the task changes

Remark 1.1 (Forward search is unavailable). Predator_1's brute-force benchmark is the canonical rule-enumeration machine: adjoin every direct consequence, stage by stage. Realising it needs an executable D_F , which the propositional system provides because its formulas are short strings and its rules are three in number.

`set.mm` provides neither. Its inference step is substitution into a schematic assertion under a variable-disjointness discipline, and its formula space is unbounded. One cannot enumerate $D_F(\Gamma)$ for a ZFC state, so neither prover in the Predator_1 comparison exists here.

Remark 1.2 (What the corpus does supply). A Metamath proof is a list of label references. The statement `thm $p - ... = L1L2....` names exactly the axioms and earlier theorems it uses, so the premise relation is not inferred but read. That is the structure Predator_2 learns, and it is the structure a prover inside a large library must guess: Sledgehammer and its descendants exist because selecting from a library is the bottleneck, not applying a rule once selected.

2 The prover

Definition 2.1 (Predator_2). Let $C = (t_0, t_1, \dots)$ be a corpus in derivation order. For a goal t_i the *available set* is $\{t_j : j < i\}$, and the *true premises* are those its recorded proof cites. Predator_2, carrying the tag $\langle \text{Predator_2} \rangle$, is a scoring function on pairs (goal, candidate) inducing a ranking of the available set.

The eleven features are cheap and computable from the file: symbol overlap between goal and candidate in both directions, how often the candidate has been cited so far, how far back it sits, the lengths of both statements, whether they share a rare symbol, whether the candidate is an axiom. Features are standardised before fitting and the training transform is reapplied at scoring.

Remark 2.2 (Hard negatives). Positives are a proof's actual premises. Negatives are drawn mostly from the premises of *nearby* theorems that this goal did not use. On a corpus of tens of thousands, uniformly random negatives are almost all trivially irrelevant to any given goal, so a model trained against them learns to reject the unrelated rather than to choose among the plausible — and choosing among the plausible is the entire task.

3 Scoring

Definition 3.1 (The two numbers). For a goal with true premise set G and candidate pool P , let ρ be the ranking Predator_2 induces. Then

$$\text{recall}@k = \frac{|G \cap \rho_{1..k}|}{|G|}, \quad \text{effort} = \frac{\max\{r : \rho_r \in G\}}{|P|}.$$

Effort is the depth one must read before every premise has been seen, so a selector may stop there. Brute force is the same quantity under a content-blind ordering.

Remark 3.2 (Neither number alone). A selector returning nothing has effort zero and recall zero. A selector returning the library in its given order has recall one and effort one. Only the pair distinguishes a useful ranking, exactly as solve rate and effort ratio do for Predator_1.

4 Measurements on a structural stand-in

Remark 4.1 (What was actually measured, stated first). No network was available when this was written, so `set.mm` could not be downloaded. Everything below was measured on a generated corpus of 6,014 statements built to imitate `set.mm`'s *structure*: citation following a power law, so a few lemmas are cited hundreds of times and most once or twice; statements drawn from topical vocabularies, so related theorems share symbols; premise counts with mean near three and a long tail. It is not mathematics and contains no theorems. It tests the pipeline, not the idea.

Remark 4.2 (A generated corpus can be wrong in an instructive way). The first version of the stand-in drew each theorem's premises by popularity alone, independently of the statement's content. Predator_2 then *lost* to the frequency baseline at $k = 5$ — correctly, because on such data frequency is the optimal predictor and there is no content signal to find. The corpus, not the model, was at fault: real lemmas about ordinals are cited by theorems about ordinals, and a stand-in without that correlation cannot test a content-based selector at all. The generator now assigns each theorem a topic, draws its statement mostly from that topic's symbols, and draws most of its premises from the same topic. This is recorded because the failure was a useful check: a harness that cannot distinguish a good selector from a bad one will happily report numbers.

	Predator_2	frequency	chronological
recall@1	0.002	0.027	0.000
recall@5	0.060	0.067	0.009
recall@10	0.192	0.089	0.011
recall@50	0.658	0.155	0.038
MRR	0.104	—	—
effort (fraction of pool read)	0.216	—	0.800
reduction against brute force	3.7×	—	—

Corpus 6,014 statements, $p = 0.8$ temporal split, 150 held-out goals, candidate pool 2,000, training 110,814 examples in 4.5 seconds.

Remark 4.3 (Read the top of the table before the bottom). Predator_2 is beaten by frequency at $k = 1$ and $k = 5$ and beats it by more than four times at $k = 50$. That shape is worth stating plainly rather than advertising the favourable row: the model is poor at naming the single most likely premise and good at confining the search to a few dozen candidates. For a prover that will try everything in its shortlist, the second is the useful property; for one that gets a single guess, it is not. Whether the shape survives on real mathematics is exactly what the stand-in cannot say.

5 Running it on ZFC

```
python predator2.py fetch set.mm # about 40 MB, ZFC, ~43,900 theorems
python predator2.py stats --db set.mm --limit 8000
python predator2.py train --db set.mm --limit 8000 -p 0.8
python predator2.py train --db set.mm # the whole corpus
```

`-limit` caps how many statements are read, which matters on a first run: the full corpus takes appreciably longer to parse than a fragment. `iset.mm`, intuitionistic set theory, is smaller and is the gentler starting point. Output goes to a folder stamped to the minute; set `PREDATOR_OUT` to place it, and quote paths containing spaces.

Remark 5.1 (What a real run would settle). Three things the stand-in cannot. Whether symbol overlap carries signal in a corpus where notation is genuinely shared across subject matter rather than assigned by a generator. Whether the recall shape of Section 4 — weak at the top, strong by $k = 50$ — is a property of the method or an artifact of synthetic topics. And whether the effort reduction survives a pool of forty thousand rather than two thousand, where the ratio matters far more.

6 Scope

Nothing here is a result about set theory, and nothing here has touched ZFC. The program parses `set.mm` and has been exercised on a corpus built to resemble it; the eleven features are hand-built; the baselines are frequency and recency, which are weak. What is established is that the pipeline runs, that the two-number metric behaves as intended, and that the harness can detect its own failure — as Remark 4.2 records, it did.

Acknowledgements

The author thanks AI assistants used as engineering and verification tools, in particular for identifying that the first generated corpus made the frequency baseline optimal by construction and therefore could not test the model it was built to test. All claims and final decisions remain the responsibility of the author.

References

- [1] Tenneson, B. (2026). *Predator: A Trainable Automated Theorem Prover*, Version 1.1.
- [2] Tenneson, B. (2026). *ML-SIC-ATP Theory: Learned Search Policies for Proof Search*, Version 1.1.
- [3] Megill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.
- [4] Paulson, L. C., & Blanchette, J. C. (2010). Three years of experience with Sledgehammer. *IWIL*.